



TRIBHUVAN UNIVERSITY
FACULTY OF HUMANITIES AND SOCIAL SCIENCE

Project Proposal
on
“MovieMagic: The Streaming Aspect of Cinematography”

Submitted to
Department of Computer Application
National College of Computer Studies

In partial fulfilment of the requirements of Bachelor's Degree of Computer Application

Submitted By:
Rusha Manandhar
(NCCSBCA101 / 2081)

Under the Supervision of
Sumit Ghising



Tribhuvan University
Faculty of Humanities and Social Sciences
National College of Computer Studies

Supervisor's Recommendations

I hereby recommend that this project prepared under my supervision by **Rusha Manandhar** entitled “**MovieMagic: The Streaming Aspect of Cinematography**” in partial fulfillment of the requirements for a degree of Bachelors of Computer Application is recommended for the final evaluation.

SIGNATURE

Sumit Ghising

SUPERVISOR

Faculty Member

Department of Computer Application

National College of Computer



Tribhuvan University
Faculty of Humanities and Social Sciences
National College of Computer Studies

Letter of Approval

This is to certify that this project prepared by **Rusha Manandhar** entitled “**MovieMagic: The Streaming Aspect of Cinematography**” in partial fulfillment of the requirements for the degree of Bachelors in Computer Application has been evaluated. In our opinion, it is satisfactory in scope and quality as a project for the required degree.

Signature of Supervisor Sumit Ghising Faculty Member Department of Computer Application National College of Computer Studies Paknajol, Kathmandu	Signature of HOD / Coordinator Rajan Poudel Faculty Member Department of Computer Application National College of Computer Studies Paknajol, Kathmandu
Signature of Internal Examiner	Signature of External Examiner

Abstract

MovieMagic is a movie streaming website designed to help users discover films they'll love by offering personalized recommendations and localized trends. It does this through a hybrid filtering system—combining content-based filtering (using TMDb's similar movies API) to recommend movies similar to what a user has watched, and collaborative filtering (using the Adjusted Cosine Similarity algorithm) to suggest films liked by users with similar preferences. To further enhance discovery, the platform features location-based trending, which identifies popular movies near the user by calculating distance using the Haversine formula and the browser's built-in location data. Users can also search easily with simple filters based on title, genre, or keyword. All movie information is fetched in real-time from the TMDb API, ensuring updated and reliable content. MovieMagic was built to simplify and personalize the movie-watching experience—making it easier for users to find content they care about, quickly and effortlessly.

Keywords: MovieMagic, personalized recommendations, hybrid filtering, content-based filtering, collaborative filtering, Adjusted Cosine Similarity, TMDb API, location-based trending, Haversine formula, movie discovery, movie streaming platform, user preferences, real-time data.

Acknowledgement

We would like to extend our heartfelt gratitude to our supervisor, **Mr. Sumit Ghising**, for his invaluable guidance, continuous monitoring, and unwavering motivation throughout all phases of this project. His expertise, constructive feedback, and insightful suggestions were instrumental in shaping the direction and success of this endeavour. We deeply appreciate his patience, support, and the inspiration he provided, which helped us overcome various challenges and achieve our goals. We also wish to express our sincere appreciation to all other supervisors, professors, and mentors who offered their time, knowledge, and encouragement. Their collective support and advice were crucial in refining our approach and enhancing the quality of our work. Additionally, we are grateful to our friends, peers, and family members for their constant encouragement, moral support, and willingness to assist whenever needed. This project would not have been possible without the collaborative efforts and contributions of everyone involved, and we are truly thankful for their role in making it a success.

Rusha Manandhar

Table of Content

Abstract	iv
Acknowledgement	v
List of Abbreviation	viii
List of Figures	ix
List of Table	x
Chapter 1 Introduction	1
1.1 Introduction	1
1.2 Problem Statement	1
1.3 Objective	1
1.4 Scope and Limitation	2
1.4.1 Scope	2
1.4.2 Limitation	2
1.5 Development Methodology	2
1.6 Report Organization	3
1.6.1 Introduction	3
1.6.2 Background Study and Literature Review	3
1.6.3 System Analysis and Design	3
1.6.4 Implementation and Testing	3
1.6.5 Conclusion and Future Recommendations	3
Chapter 2 Background Study and Literature Review	4
2.1 Background Study	4
2.2 Study of Existing Systems	4
2.3 Literature Review	4
Chapter 3 System Analysis and Design	6
3.1 System Analysis	6
3.1.1 Requirement Analysis	6

3.1.2 Feasibility Study	7
3.1.3 Data Modelling: Entity Relation Diagram	8
3.1.4 Data Modelling: Data Flow Diagram	9
3.2 System Design	11
3.2.1 Architecture Design	11
3.2.2 Database Schema Design	12
3.2.3 Interface Diagram	13
3.2.4 Physical DFD	14
3.2.5 Algorithm Details	15
Chapter 4 Implementation and Testing	17
4.1 Implementation	17
4.1.1 Tools used	17
4.1.2 Implementation details of modules	17
4.2 Testing	26
4.2.1 Test Cases for Unit Testing	26
4.2.2 Test Cases for System Testing	30
Chapter 5 Conclusion and Future Recommendations	32
5.1 Lesson Learnt / Outcome	32
5.2 Conclusion	32
5.3 Future Recommendations	32

List of Abbreviation

API	Application Programming Interface
CRUD	Create, Read, Update, Delete
PHP	Hypertext Preprocessor
SQL	Structured Query Language
TMDb	The Movie Database

List of Figures

Figure 1.1 : Waterfall Model	3
Figure 3.1 : Use Case Diagram of MovieMagic	6
Figure 3.2 : Entity Relation Diagram of MovieMagic	8
Figure 3.3 : DFD Level 0 of MovieMagic	9
Figure 3.4 : DFD Level 1 of MovieMagic	10
Figure 3.5 : Three Tier Architecture	11
Figure 3.6 : Database Schema of MovieMagic	12
Figure 3.7 : Interface Diagram of MovieMagic	13
Figure 3.8 : Physical DFD of MovieMagic	14

List of Table

Table 4.1: Unit Testing of the Registration in MovieMagic	26
Table 4.2: Unit Testing of the Login in MovieMagic	27
Table 4.3: Unit Testing of the Search Functionality in MovieMagic	28
Table 4.4: Unit Testing of the Recommendation Feature in MovieMagic	29
Table 4.5: Unit Testing of the LogOut Feature in MovieMagic	30
Table 4.6: System Testing of the MovieMagic	30

Chapter 1 Introduction

1.1 Introduction

In today's digital age, the way people discover and consume movies has evolved, with streaming platforms becoming the main source of entertainment. However, the overwhelming number of available options often makes it difficult for users to find content that truly matches their interests. MovieMagic addresses this issue by offering a smart, personalized movie streaming experience powered by advanced recommendation techniques and real-time data. It features a hybrid recommendation system that combines content-based filtering—using TMDb's similar movies API to suggest films related to what a user has watched—and collaborative filtering, which applies the Adjusted Cosine Similarity algorithm to recommend movies based on the preferences of users with similar tastes. Additionally, location-based trending helps users discover popular films in their area by utilizing the Haversine formula and browser-based location detection. With a clean interface, easy-to-use search filters, and reliable movie data from the TMDb API, MovieMagic aims to make movie discovery more intuitive, personalized, and enjoyable for every user.

1.2 Problem Statement

In today's fast-growing digital world, users have access to thousands of movies across various streaming platforms, making it increasingly difficult to find content that matches their personal taste. As a result, users often spend more time searching than watching, leading to a frustrating and time-consuming experience. Additionally, most recommendation systems fail to consider both user preferences and regional trends. MovieMagic is developed to solve this problem by providing smart, real-time, and location-aware movie recommendations through hybrid filtering techniques and real-time data integration.

1.3 Objective

The main objective of MovieMagic is:

- To provide users with movie and TV show recommendations based on what they like and watch.
- To integrate location-based trending movies so users can discover popular films in their area.

- To offer a simple and fast search system that helps users find movies quickly.
- To use real-time data from reliable sources to keep movie information accurate and up-to-date.

1.4 Scope and Limitation

1.4.1 Scope

MovieMagic is designed to help users easily discover movies and TV shows by providing personalized recommendations using a combination of content-based and collaborative filtering. It fetches accurate and up-to-date movie information from TMDb and supports searching. The platform also highlights trending movies based on the user's location to offer relevant regional suggestions. Accessible through web browsers on various devices, MovieMagic focuses on delivering a smooth, user-friendly experience for browsing and discovering content. However, it does not provide direct streaming or downloading of movies, concentrating instead on improving the way users find and explore films.

1.4.2 Limitation

MovieMagic focuses mainly on helping users discover movies and TV shows through recommendations and search, but it does not offer direct streaming or downloading of content. The platform depends entirely on the TMDb API for movie data, so any limitations or changes in that service can affect MovieMagic's functionality. Location-based trending features rely on browser location services, which may not always be accurate or available if users deny permission. Additionally, the recommendation system is limited by the data it receives and may not always perfectly match user preferences. Finally, MovieMagic requires an internet connection to fetch movie information and cannot be used offline.

1.5 Development Methodology

The Waterfall Model was used as the software development methodology for this project, ensuring a structured and sequential approach to development. Following the Waterfall Model allowed for a well-structured, secure, and efficient development process. Each phase was completed before moving on to the next, reducing the risk of errors and ensuring a well-documented process.

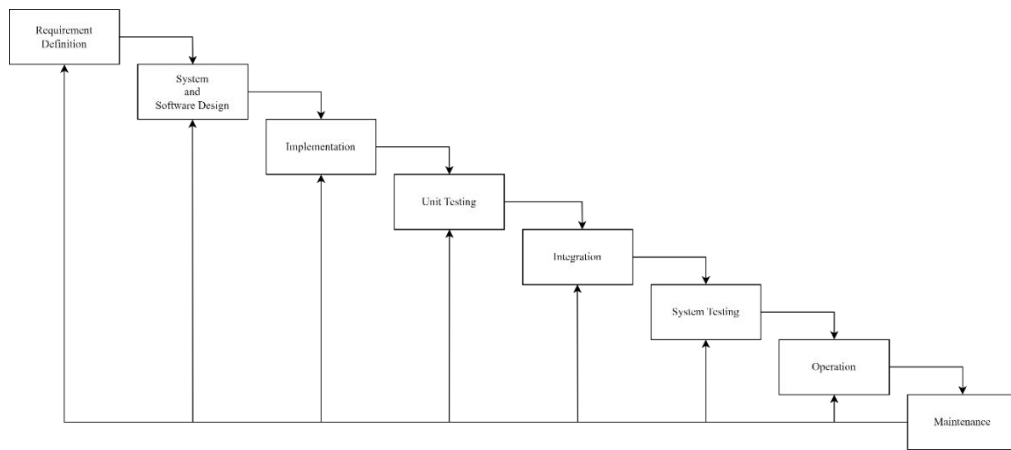


Figure 1.1: Waterfall Model

1.6 Report Organization

1.6.1 Introduction

This chapter provides an overview of the project, including the problem it aims to solve, the project objectives, scope and limitations, and development methodology of the project.

1.6.2 Background Study and Literature Review

This chapter focuses on study of existing systems and review of different articles studied. It also explores the underlying concepts and principles that informed the development of the system.

1.6.3 System Analysis and Design

This chapter covers the project's requirements analysis, feasibility assessment, and design. System Models as Data Model, Process Model and Flowcharts, functionality analysis and System Designs as Architecture Design, Database Schema Design, Interface Design and Physical DFD are all included.

1.6.4 Implementation and Testing

This chapter provides details on the project's implementation, the software and tools used, and the types of testing conducted.

1.6.5 Conclusion and Future Recommendations

This chapter discusses the project's potential outcomes and summarizes the main findings. It also provides recommendations for future work

Chapter 2 Background Study and Literature Review

2.1 Background Study

With the rise of streaming services, people have more movies and TV shows to choose from than ever before. While this gives viewers many options, it also makes it harder to find movies they really want to watch. Many platforms offer recommendations, but these are often not personalized enough or don't consider what's popular nearby. To solve this, MovieMagic was created to help users find movies and shows that match their tastes using smart recommendation methods. By combining information about what users like and what similar viewers enjoy, along with trending movies based on location, MovieMagic aims to make movie discovery easier and more fun. The platform uses data from TMDb, a popular movie database, to keep information accurate and up-to-date. Overall, MovieMagic addresses the challenge of finding the right content quickly in a growing sea of entertainment options.

2.2 Study of Existing Systems

Popular movie streaming platforms like Netflix, Amazon Prime Video, and Disney+ offer vast libraries of movies and shows with built-in recommendation systems. However, many of these platforms primarily use either content-based filtering or collaborative filtering alone, which sometimes limits the accuracy of their suggestions. For example, Netflix relies heavily on collaborative filtering but may not always consider a user's local trends or preferences in-depth. Additionally, websites like IMDb and TMDb provide extensive movie information and user ratings but don't offer personalized recommendations or location-based trending features. The search functions on these sites can sometimes be slow or complex, making it hard for users to find specific content quickly. MovieMagic improves on these systems by combining both content-based and collaborative filtering for better personalized recommendations, integrating location-based trending to highlight popular local movies, and implementing a fast and simple search system to enhance user experience.

2.3 Literature Review

The effectiveness of hybrid recommendation systems, which combine content-based and collaborative filtering techniques, has been extensively researched to improve recommendation accuracy and user satisfaction. Geetha et al. [1] demonstrated that

integrating user-based and item-based collaborative filtering with content-based approaches significantly enhances the precision and recall of recommendations. Similarly, graph-based hybrid models that incorporate user demographics and location information have been shown to address the cold-start problem effectively, thereby improving recommendation quality [2].

In addition to hybrid filtering, location-aware recommendation systems have gained importance in providing personalized content. Madadipouya [3] proposed a location-based movie recommender that leverages users' geographical data to enhance recommendation relevance. Furthermore, the Location-Aware Recommender System (LARS) developed by Microsoft Research efficiently generates scalable and accurate location-based recommendations, reinforcing the value of geographic context in content suggestion [4].

Real-time search and filtering mechanisms are critical for enhancing user experience in dynamic multimedia platforms. Recent advancements in real-time collaborative filtering have been employed to refine video search recommendations by analyzing user interaction patterns to predict content preferences more accurately [5]. Moreover, the integration of persona-based user modeling with deep learning algorithms has shown significant improvements in addressing issues such as cold start while maintaining high recommendation accuracy in real-time environments [6].

Several existing movie discovery platforms exemplify the application of these advanced techniques. FilmQuest, developed using React and Node.js, features dynamic movie search, filtering, and browsing capabilities powered by real-time data from movie APIs [7]. Similarly, MovieVault, built with Next.js 14, offers features like real-time search, bookmarking, and detailed movie information, highlighting practical implementations of hybrid recommendation systems and optimized search filtering [8].

Chapter 3 System Analysis and Design

3.1 System Analysis

3.1.1 Requirement Analysis

Requirements will be collected through personal evaluation of different existing systems, along with suggestions from mentors, and supervisors.

3.1.1.1 Functional Requirements

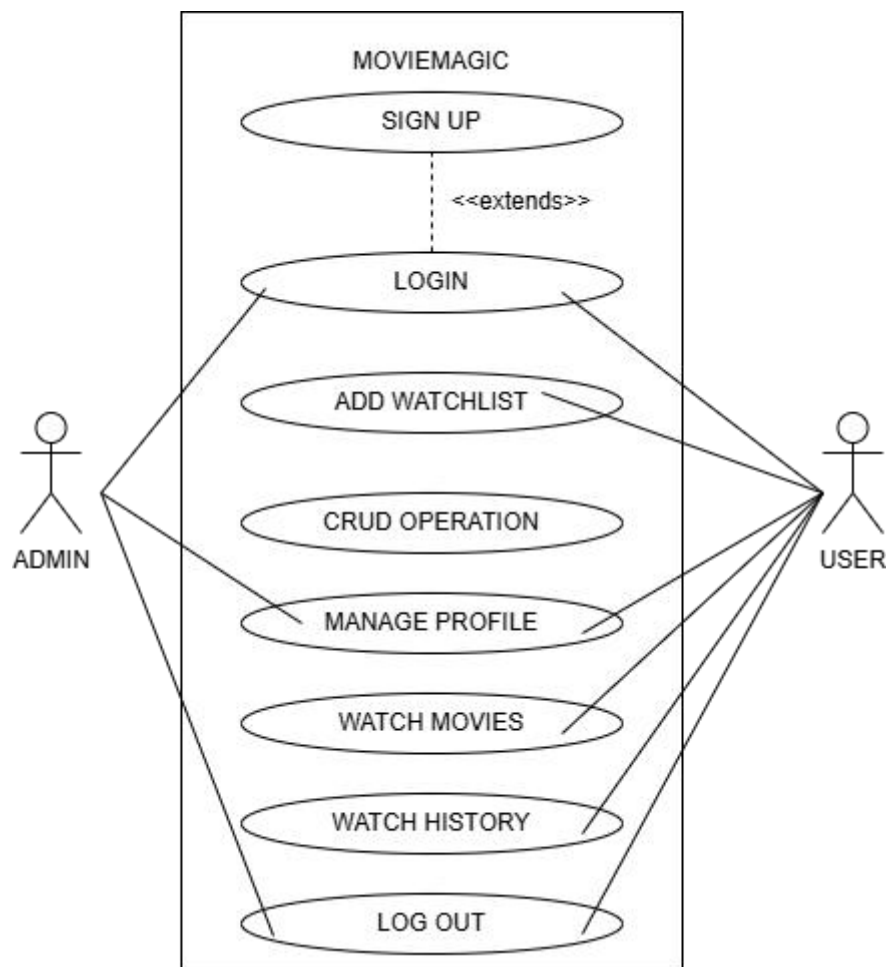


Figure 3.1: Use Case Diagram of MovieMagic

The use case diagram in previous page specifies the basic operations a user can perform in the proposed system

- A new account creation feature with a username, email and password is required.
- With their username and password, users must be able to access their accounts.
- Users with wrong username will neither be validated nor allowed to login.
- Users with correct credentials must be able to perform CRUD (Creation, Retrieve, Update and Delete) operation on the entries (Contact Support, Profile Info, Watch History and Account).
- The site should be able to stream movies/TV shows from the catalog, bookmark movies/TV shows for later viewing, auto-saves progress in watch history.
- Each watch history lists all watched content with timestamps.
- The site should include option to remove entries of watch history.
- Users should be able securely ends the user session.

3.1.1.2 Non-Functional Requirements

- It should work on phones, tablets, and computers.
- User data must be kept safe and private at all times.
- The site should be easy to use and understand for everyone.
- It should work on popular web browsers like Chrome and Firefox.
- The system should be stable and not crash often.
- The website should display clear error messages when something goes wrong
- The website should try to load results as quickly as possible

3.1.2 Feasibility Study

The system is evaluated for future development with a set of constraints. The feasibility study is done regarding the available technologies, time constraints, area of application, cost of deployment and upkeep, and future possibilities of the project.

3.1.2.1 Technical Feasibility

MovieMagic is technically feasible as it is developed using existing and well-supported technologies and tools. The frontend is built with HTML, CSS, and JavaScript, while the backend relies on the traditional web application stack, including Apache Web Server, MySQL database, and PHP. The project integrates the TMDb API to fetch real-time movie data and uses browser-based geolocation to support location-specific features. Development is carried out using Visual Studio Code as the code editor, GitHub for

version control and collaboration, and the Brave browser for testing. These platforms and technologies are widely used, open-source or freely available, and offer strong community support, making the system practical to build, deploy, and maintain within a reasonable time and cost.

3.1.2.2 Operational Feasibility

MovieMagic is operationally feasible as it runs smoothly on common web browsers like Brave, Chrome, and Firefox without needing high-end devices. It only requires a stable internet connection to fetch movie data and location services. The platform is easy to maintain using Visual Studio Code and GitHub, and can be hosted on any standard server with Apache, MySQL, and PHP support.

3.1.2.3 Economic Feasibility

MovieMagic is economically feasible as it uses open-source and free technologies like HTML, CSS, JavaScript, PHP, MySQL, and the TMDb API, which eliminates licensing costs. Development tools such as Visual Studio Code, GitHub, and Brave browser are also free to use. Since the platform can be hosted on low-cost servers, the overall development and maintenance expenses remain minimal and manageable.

3.1.3 Data Modelling: Entity Relation Diagram

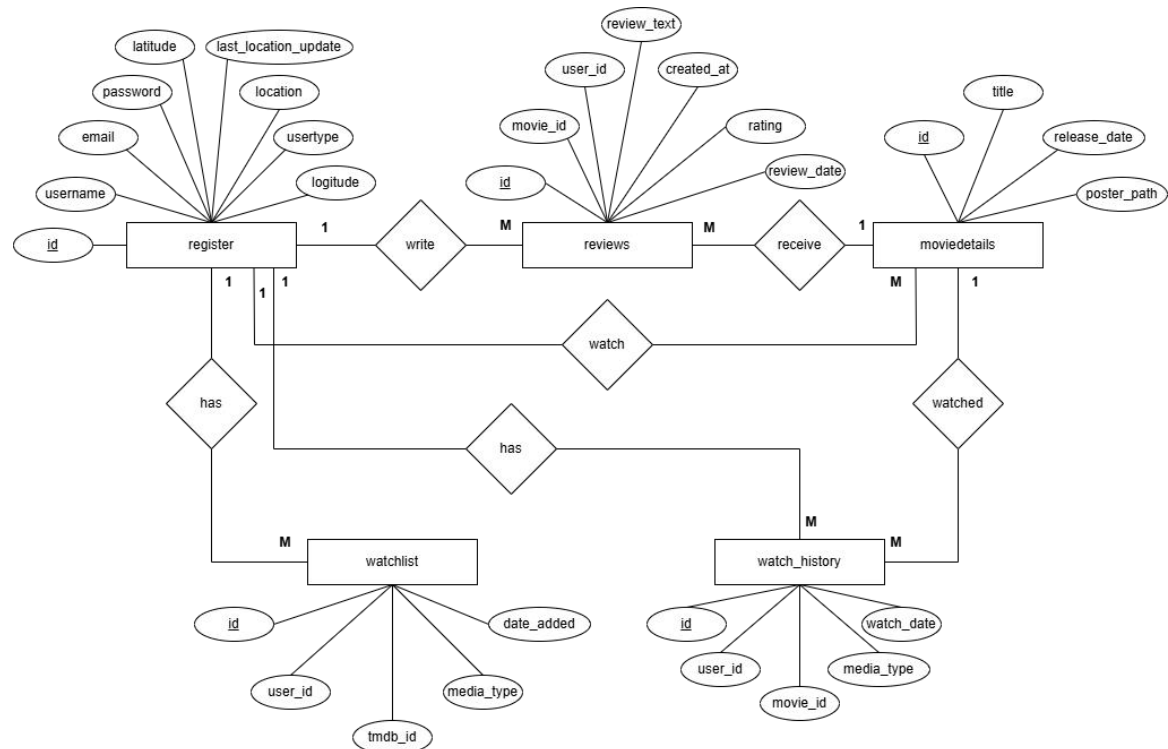


Figure 3.2: Entity Relation Diagram of MovieMagic

The above table represents the following relation:

- One-to-many relationship between “**Register**” and “**Reviews**” entity where one user can write multiple reviews..
- One-to-many relationship between “**Register**” and “**Watchlist**” entity where one user can save multiple movies/TV shows to their watchlist.
- One-to-many relationship between “**Register**” and “**Watch_history**” entity where one user can have multiple entries in their watch history.
- One-to-many relationship between “**Moviedetails**” and “**Reviews**” entity where one movie can receive multiple reviews from users.
- One-to-many relationship between “**Moviedetails**” and “**Watch_history**” entity where one movie can be watched by multiple users.
- One-to-many relationship between “**Moviedetails**” and “**Register**” entity where a user can watch many movies.

3.1.4 Data Modelling: Data Flow Diagram

i. DFD Level 0

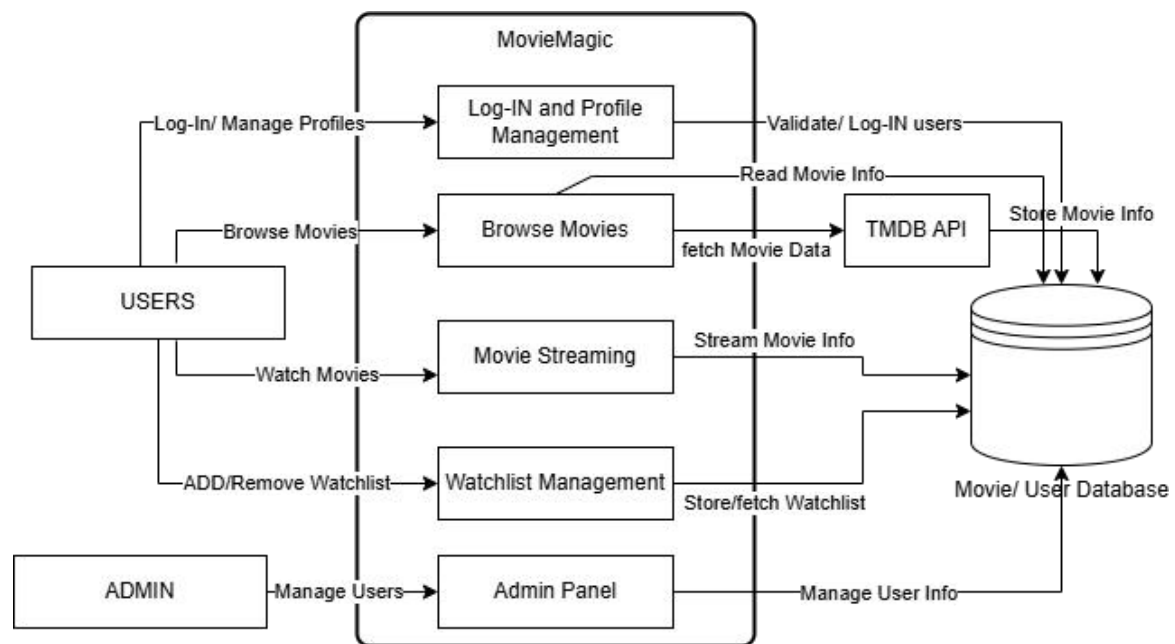


Figure 3.3: DFD Level 0 of MovieMagic

ii. DFD Level 1

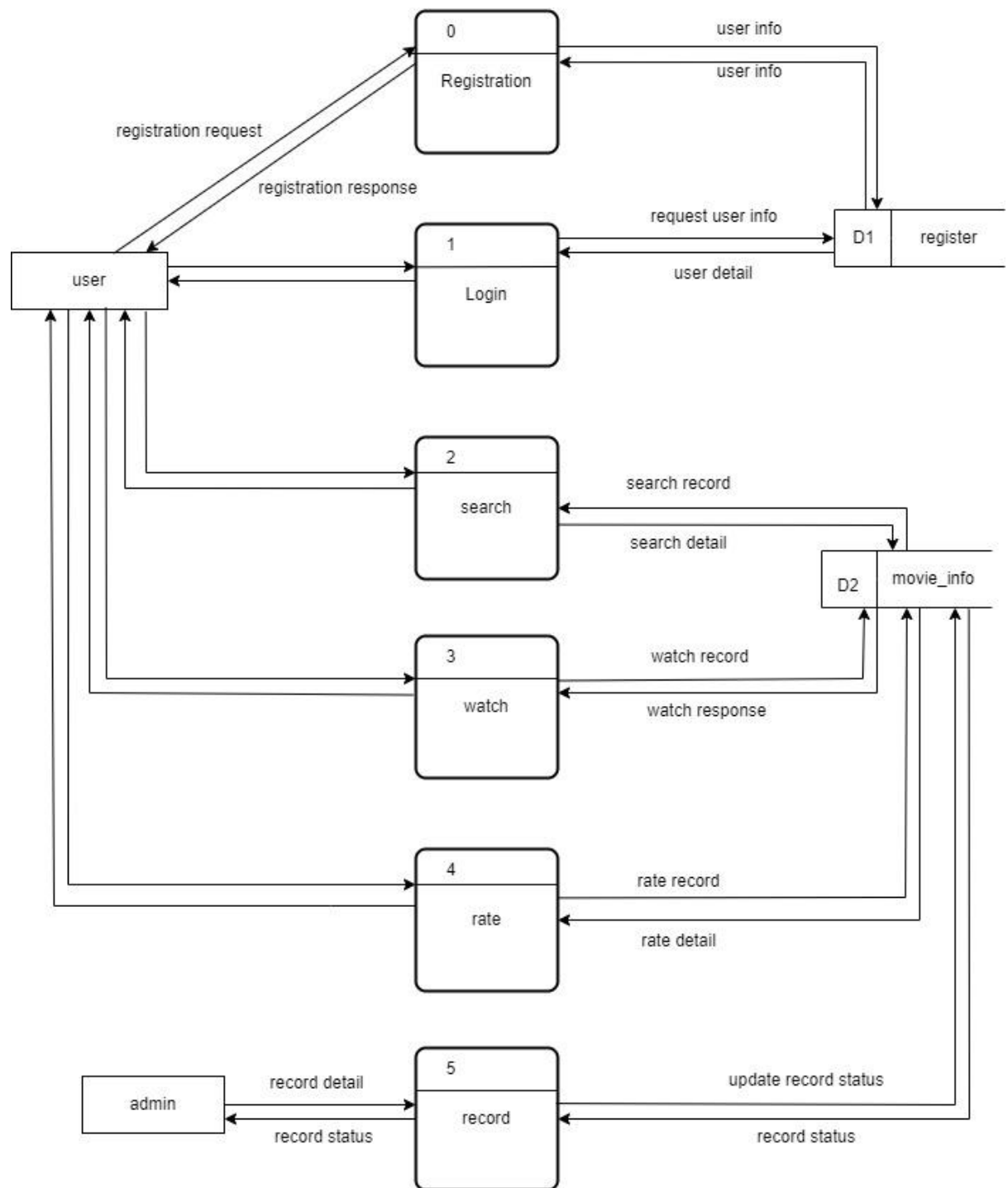


Figure 3.4: DFD Level 1 of MovieMagic

The data flow in the "MovieMagic" system includes users register or log in, storing their details in the database. They can browse movies, which pulls data from the system's catalog, and save favorites to their watchlist. When they watch or rate a movie, it updates their history and reviews. Admins can manage all users. The system also fetches movie data from external sources like TMDB. Everything is securely stored in a MySQL database.

3.2 System Design

3.2.1 Architecture Design

The three-tier architecture has been used for this project. This architecture is a client-server architecture in which the functional process logic, data access, computer data storage and user interface are developed and maintained as independent modules on separate platforms.

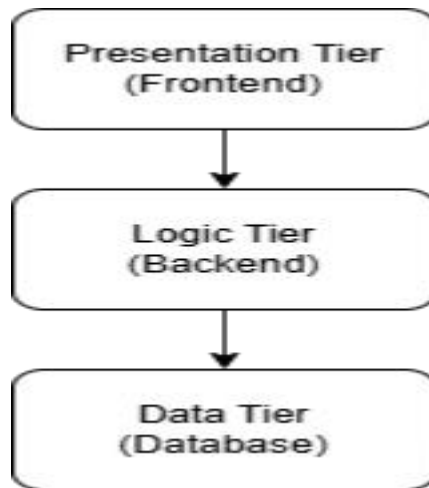


Figure 3.5: Three Tier Architecture

Three-Tier Architecture has the following layers.

- **Client Layer:** The Presentation Layer is the front-end of MovieMagic, built with HTML, CSS, and JavaScript. It allows users to browse, search, log in, view recommendations, and manage watchlists. This layer handles user input and displays data from the server with features like real-time search and easy navigation.
- **Application Layer:** The Logic Layer, built with PHP, connects the frontend and backend by managing user authentication, processing searches, handling watch history and watchlists, and integrating with the TMDb API for movie data. It generates personalized recommendations using hybrid filtering and ensures security, efficient performance, and proper routing of dynamic content.
- **Database Layer:** The Data Layer handles storage and retrieval using MySQL, hosted on the Apache Web Server. It stores user accounts, movie details, reviews, watch history, watchlists, and search data. This layer ensures secure, consistent, and reliable data handling, responding to logic-tier queries with structured data for display.

Following are the reasons behind choosing three tier-architecture:

- An ability to separate the user interface, business logic, and data storage into distinct layers, making the system easier to develop, test, maintain, and scale.

3.2.2 Database Schema Design

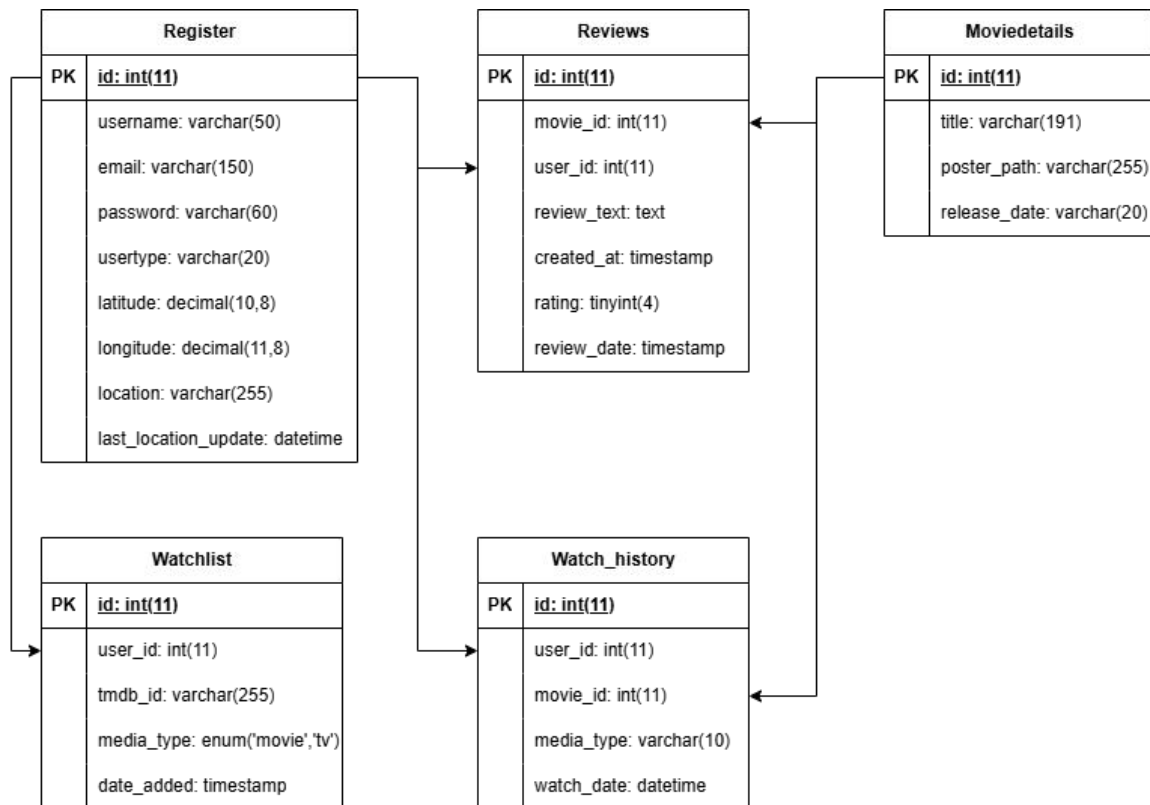


Figure 3.6: Database Schema of MovieMagic

The database schema design for MovieMagic includes the following tables:

1. **register**: This table stores user information, including the 'id', 'username', 'email', 'password', 'latitude', 'longitude', 'location' and 'last_location_update' datetime.
2. **reviews**: This table store the information about user generated ratings and text reviews, including the 'user_id' and 'movie_id' as foreign keys.
3. **moviedetails**: This table stores core movie/TV show metadata (title, release date, poster URL). It includes relationships like; 1-to-many with reviews (movies receive multiple reviews). 1-to-many with watch_history (movies appear in multiple view records).
4. **watchlist**: This table tracks user-saved "to watch" items, including user_id and tmdb_id(external tmdb api reference) as it's foreign keys.
5. **watch_history**: This table logs view content with timestamps. The user_id and movie_id fields acts as a foreign key constraints

3.2.3 Interface Diagram

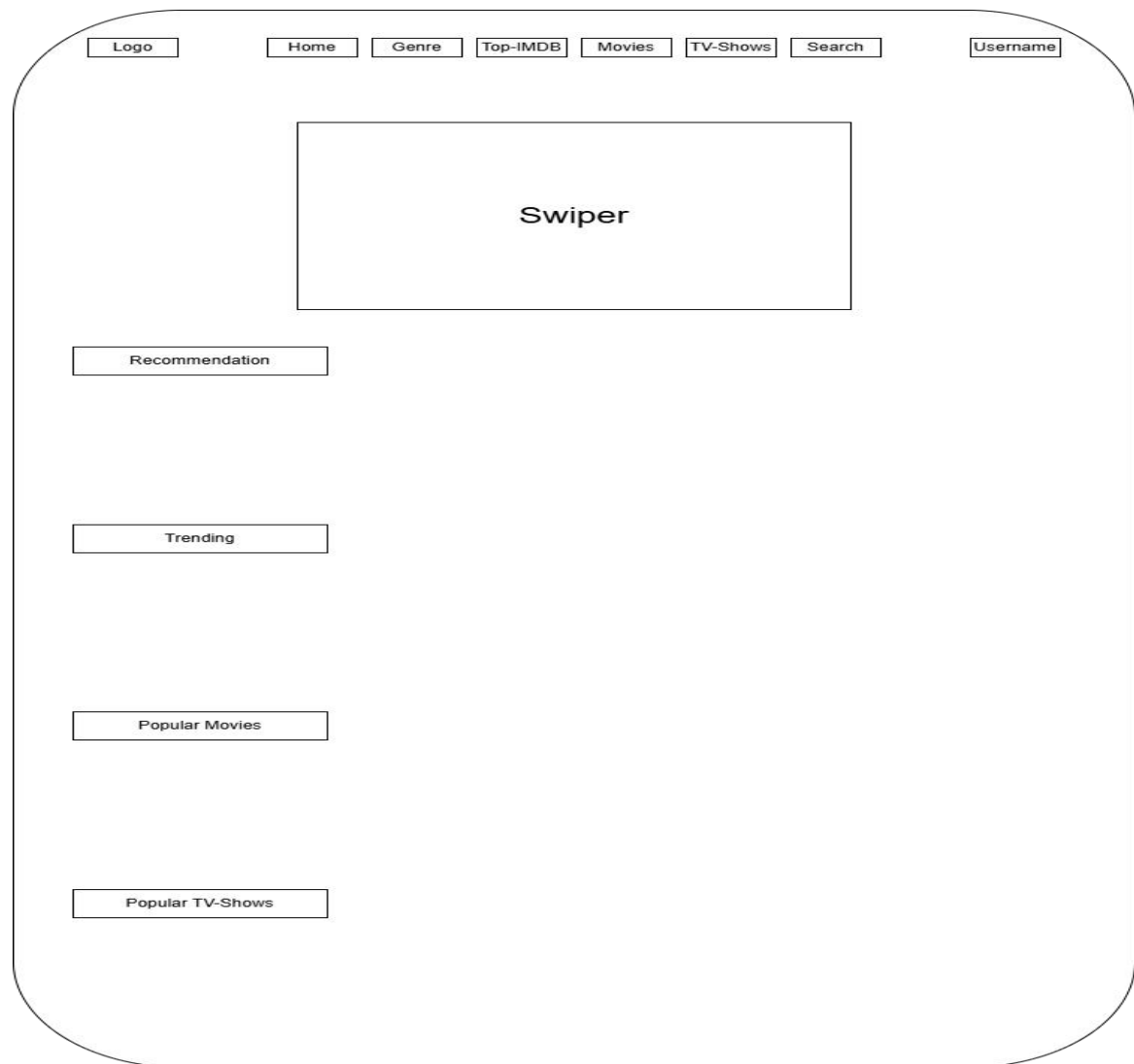


Figure 3.7: Interface Diagram of MovieMagic

The interface structure design defines the basic components of the interface and how they work together to provide functionality to users. An interface structure diagram (ISD) is used to show how all the screens, forms, and reports used by the system are related and how the user moves from one to another.

3.2.4 Physical DFD

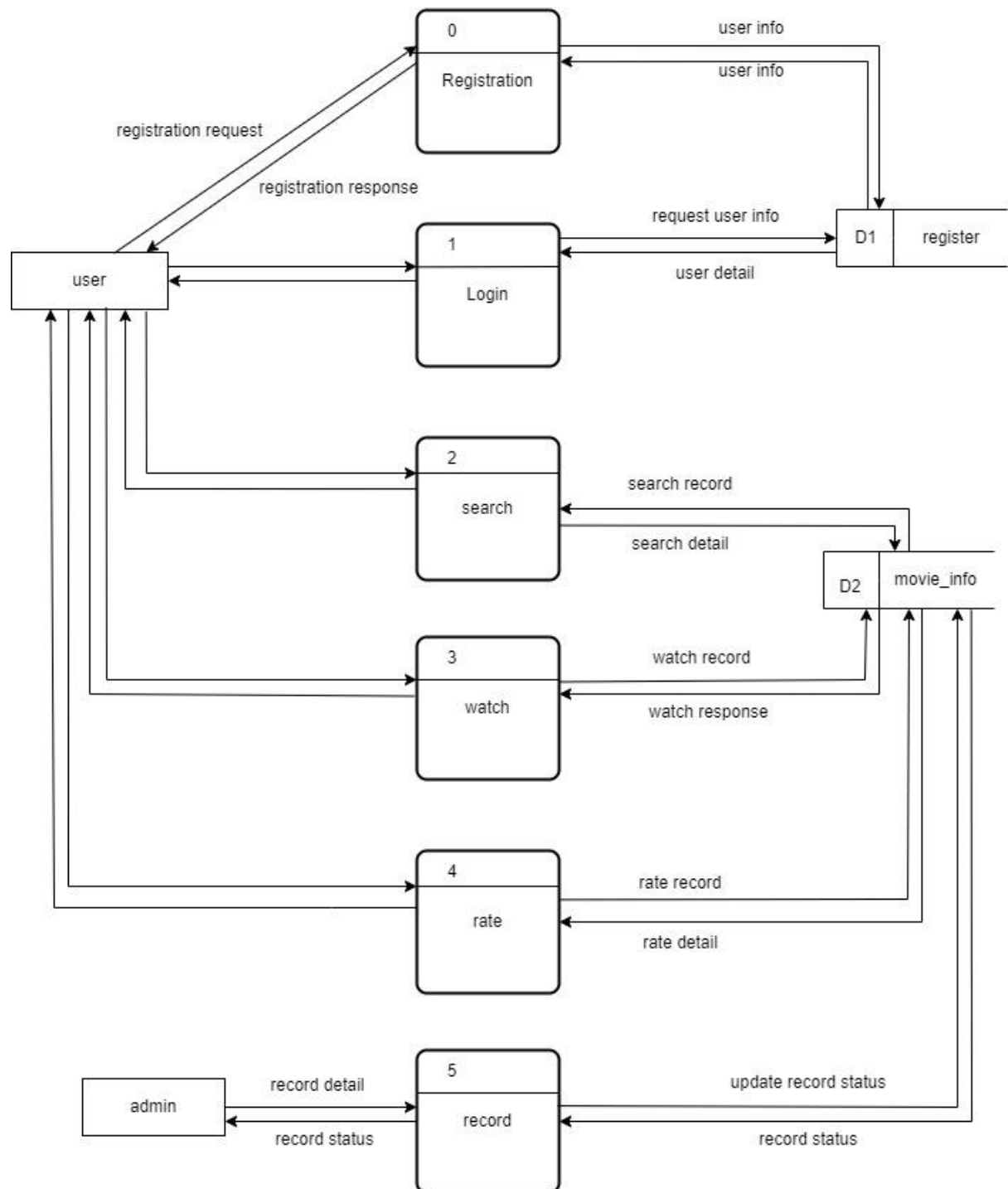


Figure 3.8: Physical DFD of MovieMagic

The above figure is a Physical DFD that visualizes user's activity, and the flow of the data between the user, and MySQL database. The user performs activities as logging in, searching, watching, rating, updating, checking history, and inserting favorites to their watchlist.

3.2.5 Algorithm Details

The MovieMagic platform implements a hybrid recommendation system that combines content-based and collaborative filtering approaches to provide personalized movie and TV show recommendations to users.

The recommendation comprises the following steps:

1. Hybrid Recommendation Approach: The system uses a weighted combination of two recommendation methods: Content-based filtering(70% weight) and Collaborative filtering(30% weight)
2. Content-based Filtering:
 - The first round of content-based filtering involves tracking users watch history, records movie IDs and media type and stores user rating for watched content
 - The second round uses TMDb API to fetch similar content for each watched item, considers movie metadata and attributes, then normalizes scores to a 0-1 scale
 - The third round scores formula(score=0.7 *(vote_average/10))
3. Collaborative Filtering:
 - The first round of Collaborative Filtering involves using adjusted cosine similarity which requires minimum of 2 common rated movies between users with Similarity threshold: 0.3
 - The second round checks the similarity using Similarity formula(similarity = $(pSum - (sum1 * sum2 / n)) / \sqrt{((sum1Sq - pow(sum1, 2) / n) * (sum2Sq - pow(sum2, 2) / n))}$)
 - Where:
 - ◆ pSum: Sum of product of ratings
 - ◆ sum1, sum2: Sum of ratings for each user
 - ◆ sum1Sq, sum2Sq: Sum of squared ratings
 - ◆ n: Number of common movies
 - The third round generates the recommendation according to the Aggregates ratings from similar users which Requires minimum of 2 similar users and than it Normalizes scores to 0-1 scale using scoring formula $(0.3 * (avg_rating / 5))$.
4. Recommendation Process Flow:

- It initially verifies if the user has watched any content or not through users watch history and ratings.
 - For, Content-based filtering it fetches similar content for each watched item and excludes already watched content. Then it calculates content-based scores.
 - For, Collaborative filtering it firstly, identifies similar users based on rating patterns and collects highly-rated content from similar users and excludes user's watched content. Then at the end it calculates collaborative scores
5. Output: the output of Hybrid Filtering is it combines both recommendation type sort by combined score and returns top 12 recommendations.

In summary, The Hybrid Approach provides a robust foundation for personalized content discovery. The combination of content-based and collaborative filtering methods ensures diverse and relevant recommendations while maintaining scalability and performance.

Chapter 4 Implementation and Testing

4.1 Implementation

4.1.1 Tools used

The development of MovieMagic involved the use of several tools and technologies to ensure an efficient and responsive system. Visual Studio Code was used as the primary code editor for writing frontend and backend code. The local development environment was set up using XAMPP, which includes Apache for hosting the server, MySQL for database management, and PHP for backend scripting. The TMDb API was integrated to fetch real-time movie and TV show data, enhancing the platform's content variety. For version control and collaboration, Git and GitHub were used. The system was frequently tested on the Brave browser to ensure a seamless user experience. PHPMyAdmin helped in managing and verifying database operations, and diagramming tools such as Canva and Draw.io were employed to design system diagrams including DFDs, architectural diagrams, and database schemas.

4.1.2 Implementation details of modules

In this section, we provide a detailed description of the procedures, functions, classes, and methods used in the implementation of the system modules.

Module 1: Location-Based Content Discovery Module

The location-based content discovery module in MovieMagic helps identify users who are geographically close to each other by using the Haversine formula. It retrieves the current user's latitude and longitude from the database, then compares this with the coordinates of other users. If another user is within a 50 km radius, their ID is stored in a list of nearby users. This feature can be used to personalize content, enable local trends, or build community-based recommendations.

Implementation:

```
// Location Data Retrieval
$location_query = "SELECT latitude, longitude FROM register WHERE id = ?";
$location_stmt = $connection->prepare($location_query);
$location_stmt->bind_param("i", $user_id);
$location_stmt->execute();
$location_result = $location_stmt->get_result();
$user_location = $location_result->fetch_assoc();
```

```

// Haversine Formula Implementation
function calculateDistance($lat1, $lon1, $lat2, $lon2) {
    $earthRadius = 6371; // Earth's radius in kilometers

    $lat1 = deg2rad($lat1);
    $lon1 = deg2rad($lon1);
    $lat2 = deg2rad($lat2);
    $lon2 = deg2rad($lon2);

    $dlat = $lat2 - $lat1;
    $dlon = $lon2 - $lon1;

    $a = sin($dlat/2) * sin($dlat/2) + cos($lat1) * cos($lat2) * sin($dlon/2) * sin($dlon/2);
    $c = 2 * atan2(sqrt($a), sqrt(1-$a));
    $distance = $earthRadius * $c;

    return $distance;
}

// Nearby Users Detection
$nearby_users = [];
if ($user_location && $user_location['latitude'] && $user_location['longitude']) {
    $users_query = "SELECT id, latitude, longitude FROM register WHERE latitude IS
NOT NULL AND longitude IS NOT NULL AND id != ?";
    $users_stmt = $connection->prepare($users_query);
    $users_stmt->bind_param("i", $user_id);
    $users_stmt->execute();
    $users_result = $users_stmt->get_result();

    while ($user = $users_result->fetch_assoc()) {
        $distance = calculateDistance(
            $user_location['latitude'],
            $user_location['longitude'],

```

```

        $user['latitude'],
        $user['longitude']
    );

    if ($distance <= 50) { // Within 50km
        $nearby_users[] = $user['id'];
    }
}
}
}

```

Module 2: Hybrid Recommendation Engine Module

This recommendation module in MovieMagic combines content-based and collaborative filtering to suggest movies. Content-based filtering (70% weight) fetches similar movies from TMDb based on what the user has already watched. Collaborative filtering (30% weight) compares the user's ratings with those of other users to find similar taste patterns and recommend movies accordingly. The system blends both methods for personalized and relevant suggestions.

Implementation

```

// Content-Based Recommendations (70% weight)
$content_recommendations = [];
foreach ($watched_movies as $watched) {
    $similar_url =
"$tmdb_base_url/{$watched['media_type']}/{ $watched['id']}/similar?api_key=$tmdb_api_key";
    $similar_data = json_decode(file_get_contents($similar_url), true);

    if (isset($similar_data['results'])) {
        foreach ($similar_data['results'] as $movie) {
            if (!in_array($movie['id'], $watched_ids)) {
                $key = $movie['id'] . '_' . $watched['media_type'];
                $content_recommendations[$key] = [
                    'id' => $movie['id'],
                    'title' => $movie['title'] ?? $movie['name'],
                    'poster_path' => $movie['poster_path'],
                    'vote_average' => $movie['vote_average'],
                    'media_type' => $watched['media_type'],
                    'score' => 0.7 * ($movie['vote_average'] / 10)
                ];
            }
        }
    }
}
}
}

```

```

// Collaborative Filtering (30% weight)
if (count($user_ratings) >= 3) {
    $similar_users = [];
    foreach ($other_users as $other_user) {
        $common_movies = array_intersect(
            array_keys($user_ratings),
            array_keys($other_ratings)
        );

        if (count($common_movies) >= 2) {
            $similarity = calculateUserSimilarity(
                $user_ratings,
                $other_ratings,
                $common_movies
            );

            if ($similarity > 0.3) {
                $similar_users[$other_user_id] = $similarity;
            }
        }
    }
}

```

Module 3: Advance Validation and Security Module

This User Authentication and Security Validation Module in MovieMagic is a comprehensive system ensuring secure user interactions and safe data handling. It includes session validation to prevent hijacking and automatic logouts on inactivity, input validation for usernames, emails, and passwords, and CSRF protection to prevent cross-site request forgery. It also features API key validation, rate limiting to avoid abuse, file upload filtering, location data checks, and content sanitization for protection against SQL injection and XSS attacks. Additionally, it logs system errors and enforces security headers to harden the application against common web vulnerabilities.

Implementation:

```

// User Authentication Validation
<?php
class ValidationModule {
    private $connection;
    private $errors = [];

    public function __construct($connection) {
        $this->connection = $connection;
    }
}

```

```
}
```

```
// Session Validation
```

```
public function validateSession() {  
    if (!isset($_SESSION['user_username'])) {  
        header("Location: userlogin.php");  
        exit();  
    }  
}
```

```
// Session Hijacking Prevention
```

```
if (!isset($_SESSION['last_activity']) ||  
    (time() - $_SESSION['last_activity'] > 1800)) {  
    session_unset();  
    session_destroy();  
    header("Location: userlogin.php?error=session_expired");  
    exit();  
}  
$_SESSION['last_activity'] = time();  
}
```

```
// Input Validation
```

```
public function validateInput($data, $type) {  
    switch($type) {  
        case 'email':  
            return filter_var($data, FILTER_VALIDATE_EMAIL);  
        case 'username':  
            return preg_match('/^[a-zA-Z0-9_]{3,20}$/', $data);  
        case 'password':  
            return strlen($data) >= 8 &&  
                preg_match('/[A-Z]/', $data) &&  
                preg_match('/[a-z]/', $data) &&  
                preg_match('/[0-9]/', $data);  
        case 'movie_id':  
            return is_numeric($data) && $data > 0;  
    }  
}
```

```

        default:
            return false;
    }
}

// CSRF Protection
public function generateCSRFToken() {
    if (!isset($_SESSION['csrf_token'])) {
        $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
    }
    return $_SESSION['csrf_token'];
}

public function validateCSRFToken($token) {
    return isset($_SESSION['csrf_token']) &&
        hash_equals($_SESSION['csrf_token'], $token);
}

// API Request Validation
public function validateAPIRequest($api_key) {
    $valid_key = '99e2fa37c0f75b95a971c97b093025cc';
    return hash_equals($valid_key, $api_key);
}

// Rate Limiting
public function checkRateLimit($user_id, $action) {
    $timeframe = 3600; // 1 hour
    $max_requests = 100;

    $query = "SELECT COUNT(*) as count FROM rate_limits
        WHERE user_id = ? AND action = ?
        AND timestamp > DATE_SUB(NOW(), INTERVAL 1 HOUR)";

    $stmt = $this->connection->prepare($query);

```



```

$stmt->bind_param("is", $user_id, $action);
$stmt->execute();
$result = $stmt->get_result();
$count = $result->fetch_assoc()['count'];

return $count < $max_requests;
}

// Content Validation
public function validateContent($content) {
    // XSS Prevention
    $content = htmlspecialchars($content, ENT_QUOTES, 'UTF-8');

    // SQL Injection Prevention
    $content = $this->connection->real_escape_string($content);

    return $content;
}

// File Upload Validation
public function validateFileUpload($file) {
    $allowed_types = ['image/jpeg', 'image/png', 'image/gif'];
    $max_size = 5 * 1024 * 1024; // 5MB

    if (!in_array($file['type'], $allowed_types)) {
        $this->errors[] = "Invalid file type";
        return false;
    }

    if ($file['size'] > $max_size) {
        $this->errors[] = "File too large";
        return false;
    }
}

```

```

        return true;
    }

    // Location Data Validation
    public function validateLocation($latitude, $longitude) {
        return is_numeric($latitude) &&
            is_numeric($longitude) &&
            $latitude >= -90 &&
            $latitude <= 90 &&
            $longitude >= -180 &&
            $longitude <= 180;
    }
}

// Implementation in HomePage.php
$validator = new ValidationModule($connection);

// Session Validation
$validator->validateSession();

// CSRF Protection
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    if (!$validator->validateCSRFToken($_POST['csrf_token'])) {
        die('CSRF token validation failed');
    }
}

// Rate Limiting
if (!$validator->checkRateLimit($_SESSION['user_id'], 'api_request')) {
    die('Rate limit exceeded');
}

// Content Validation
$search_query = $validator->validateContent($_GET['query'] ?? '');

```

?>

// Error Handling Implementation

<?php

class ErrorHandler {

public static function handleError(\$errno, \$errstr, \$errfile, \$errline) {

 \$error_log = "logs/error.log";

 \$timestamp = date('Y-m-d H:i:s');

 \$error_message = "[\$timestamp] Error: \$errstr in \$errfile on line \$errline\n";

 error_log(\$error_message, 3, \$error_log);

 if (ini_get('display_errors')) {

 echo "<div class='error-message'>An error occurred. Please try again later.</div>";

 }

 return true;

 }

}

set_error_handler('ErrorHandler::handleError');

?>

// Security Headers

<?php

header("Content-Security-Policy: default-src 'self'; script-src 'self' 'unsafe-inline' 'unsafe-eval'; style-src 'self' 'unsafe-inline'");

header("X-Frame-Options: DENY");

header("X-XSS-Protection: 1; mode=block");

header("X-Content-Type-Options: nosniff");

header("Referrer-Policy: strict-origin-when-cross-origin");

?>

4.2 Testing

4.2.1 Test Cases for Unit Testing

Project Name: MovieMagic

i. Unit Testing of the Registration in MovieMagic

Test Case ID: 1

Test Title: Verify registration with valid and invalid data

Test Designed Date: 27 Mar, 2025

Test Summary: Unit test for the user registration functionality in MovieMagic.

Pre-conditions: User is on the registration page.

Table 4.1: Unit Testing of the Registration in MovieMagic

Test Steps	Test Data	Expected Result	Actual Result	Status (Pass/Fail)
1. Navigate to registration page 2. Fill form 3. Submit	Username: movielover Email: <u>love@movie.com</u> Password: Movie@123 Re-password: Movie@123	Redirect to login page	Redirect to login page	Pass
1. Navigate to registration page 2. Submit Empty Form	Username: Email: Password: Re-password:	Displays “Please fill out this fields”	Displays “Please fill out this fields”	Pass

Post-condition:

Account is created and redirected to login.

ii. Unit Testing of the Login in MovieMagic

Test Case ID: 2

Test Title: Verify login functionality

Test Designed Date: 27 Mar, 2025

Test Summary: Conduct unit testing for the login process within MovieMagic to ensure its functionality.

Pre-conditions: User is already registered

Table 4.2: Unit Testing of the Login in MovieMagic

Test Steps	Test Data	Expected Result	Actual Result	Status (Pass/Fail)
1.Navigate to login page 2. Enter credentials 3. Submit	Username: movielover Password: Movie@123	Redirect to homepage	User navigated to homepage	Pass
1. Submit with invalid credentials	Username: wrong Password: wrong	Show error “Invalid Username or Password”	Error displayed	Pass

Post-condition:

User is authenticated and session started.

iii. Unit Testing of the Search Functionality in MovieMagic

Test Case ID: 3

Test Title: Validate movie search functionality

Test Designed Date: 27 Apr, 2025

Test Summary: To verify that the search functionality retrieves results accurately based on user input.

Pre-conditions: User is logged in

Table 4.3: Unit Testing of the Search Functionality in MovieMagic

Test Steps	Test Data	Expected Result	Actual Result	Status (Pass/Fail)
1. Enter a valid movie name in search bar 2. Press Search	Query: Inception	Show search results	Search results shown	Pass
1. Enter random characters 2. Press Search	Query: xyzabc123	Show “No results found”	Proper message displayed	Pass

Post-condition:

Search results are displayed or an appropriate message is shown for invalid input.

iv. Unit Testing of the Recommendation Feature in MovieMagic

Test Case ID: 4 Test Title: Validate hybrid recommendation system Test Designed Date: 27 Apr, 2025 Test Summary: To ensure the recommendation engine provides personalized movie suggestions based on watch history and preferences.

Pre-conditions: User has watched at least one movie.

Table 4.4: Unit Testing of the Recommendation Feature in MovieMagic

Test Steps	Test Data	Expected Result	Actual Result	Status (Pass/Fail)
1. Watch 3+ movies 2. Go to recommendation page	Watched IDs: 123, 456, 789	Show relevant movie recommendations	Recommendations displayed	Pass
1. Login as a new user 2. Visit recommendation section		Show trending or default popular movies	Trending list shown	Pass

Post-condition: Movies are suggested based on content and collaborative filtering logic.
--

v. Unit Testing of the LogOut Feature in MovieMagic

Test Case ID: 5

Test Title: Verify User Logout

Test Designed Date: 27 Apr, 2025

Test Summary: To verify that users can securely log out of the system.

Pre-conditions: User is logged in.

Table 4.5: Unit Testing of the LogOut Feature in MovieMagic

Test Steps	Test Data	Expected Result	Actual Result	Status (Pass/Fail)
1. Click on Logout		Redirect to login page	User logged out	Pass

Post-condition:

User session is destroyed and the user is returned to the login screen.

4.2.2 Test Cases for System Testing

i. System Testing of the MovieMagic(End-to-End)

Test Case ID: 1

Test Title: End-to-End Workflow of MovieMagic

Test Designed Date: 08 May, 2025

Test Summary: To verify that the full user flow—registration, login, browsing, watching, recommending, and logout—works seamlessly in MovieMagic

Pre-conditions: To verify that the full user flow—registration, login, browsing, watching, recommending, and logout—works seamlessly in MovieMagic

Table 4.6: System Testing of the MovieMagic

Test Description	Test Data	Expected Result	Actual Result	Status (Pass/Fail)
Registration	Username: moviefan Email: fan@movie.com	Redirect to login page	Success	Pass

	Password: MovieFan@123			
Login	Username: moviefan Password: MovieFan@123	Redirect to homepage	Success	Pass
Search Movie	Query: Avatar	Display matching result	Success	Pass
Watch movie (track watched status)	Movie ID: 2023	Movie marked as watched	Tracked	Pass
Generate Recommendations	Based on watched history	Show personalized list	Shown	Pass
LogOut		Redirect to login Page	Success	Pass

Post-condition:

User completes a full session through the main MovieMagic features successfully, confirming end-to-end system functionality.

Chapter 5 Conclusion and Future Recommendations

5.1 Lesson Learnt / Outcome

From this project, I learned about importance of time management while doing a project, how the coding structure (i.e, the folders structure) is maintained and how programming is done on the projects. I also learned to build the diagrams that are needed for the system. I learned many things which can lead us to do any work related to projects further.

5.2 Conclusion

MovieMagic provides a smooth and secure movie streaming experience with features like user authentication, personalized recommendations, search optimization, and multilingual support. Its robust validation and security modules ensure data integrity and user safety. Thorough testing confirms the system's reliability, making MovieMagic a well-rounded platform with strong potential for future enhancements.

5.3 Future Recommendations

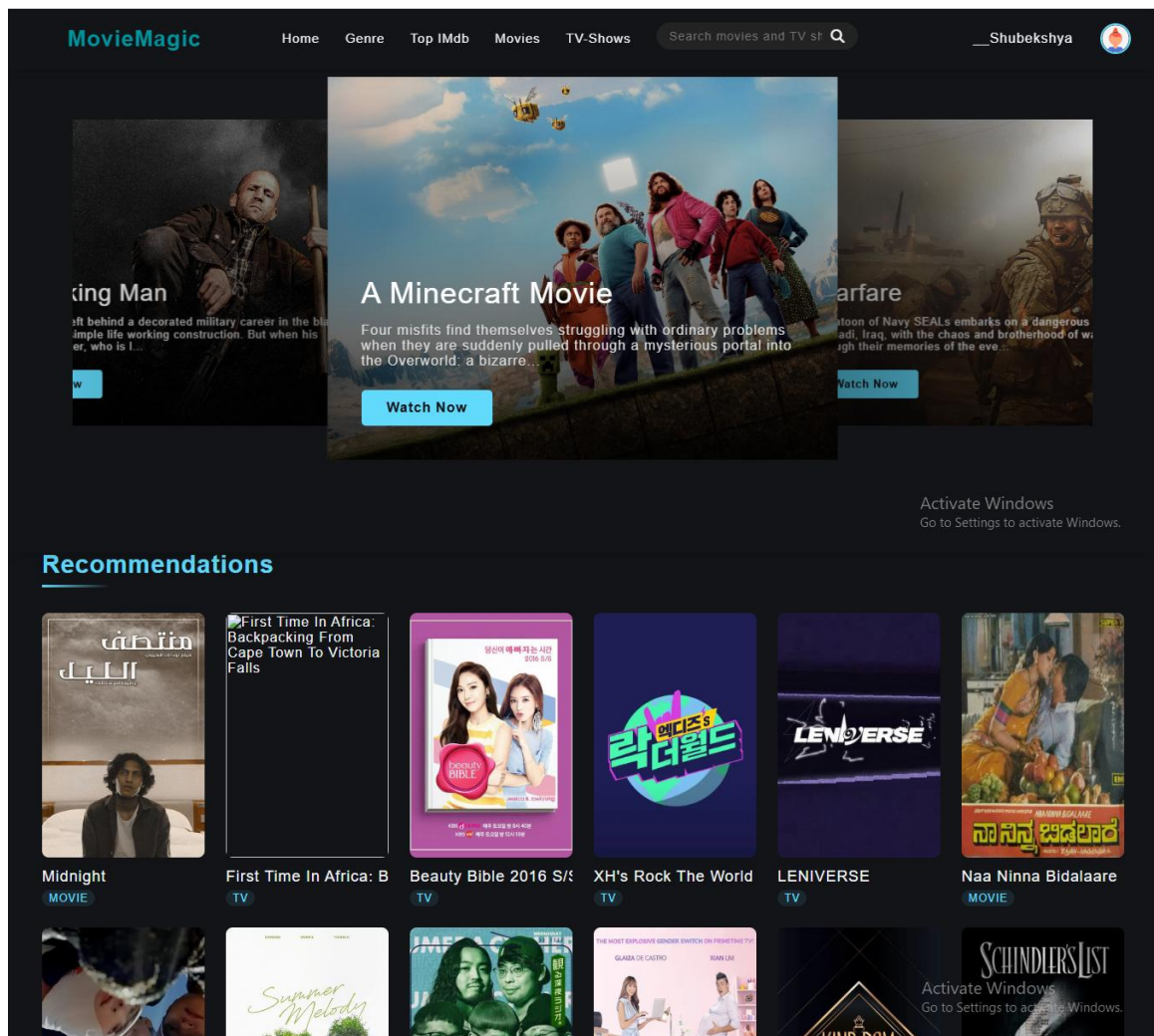
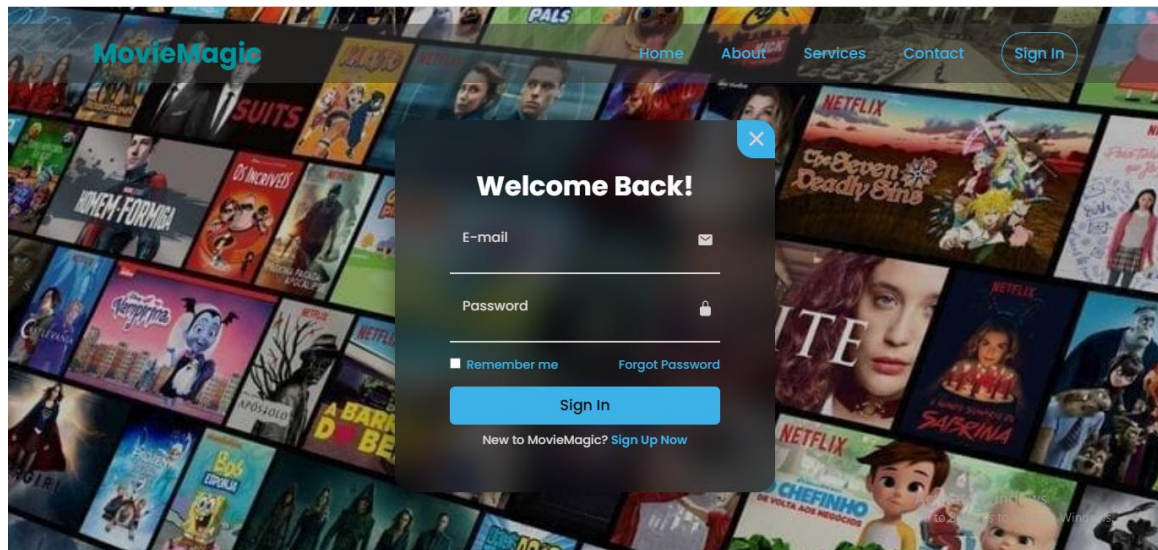
The project will be upgraded and extended with the following features:

- Create a dedicated Android and iOS application for wider reach.
- Allow users to download movies for offline viewing.
- Offer tiered plans with added benefits like early access or ad-free content.
- Allow sharing watchlists and reviews on platforms like Instagram or Twitter.

References

- [1] Geetha, S., & Surya, R. (2020). Hybrid Recommender Systems: Combining Collaborative and Content-Based Filtering. *International Journal of Computer Applications*, 975, 8887.
- [2] Zhang, S., Yao, L., Sun, A., & Tay, Y. (2019). Deep Learning Based Recommender System: A Survey and New Perspectives. *ACM Computing Surveys (CSUR)*, 52(1), 1–38.
- [3] · Madadipouya, K. (2016). A location-based movie recommender system using geospatial data. *International Journal of Computer Applications*, 134(5), 7–10.
- [4] Berjani, B., & Strufe, T. (2011). A Recommendation System for IPTV Using Social Networks. In *Proceedings of the 4th ACM Conference on Recommender Systems* (pp. 285–288). ACM.
- [5] Aggarwal, C. C. (2016). *Recommender Systems: The Textbook*. Springer.
- [6] Li, J., Kawale, J., & Fu, Y. (2015). Deep Collaborative Filtering via Marginalized Denoising Auto-encoder. In *Proceedings of the 24th ACM International on Conference on Information and Knowledge Management* (pp. 811–820).
- [7] FilmQuest. (2023). Open-source movie search and recommendation platform. GitHub Repository. Retrieved from <https://github.com/username/filmquest>
- [8] MovieVault. (2024). Real-time movie discovery platform with Next.js 14. GitHub Repository. Retrieved from <https://github.com/username/movievault>

Appendix



Trending



Your Watch History

Movies and shows you've watched

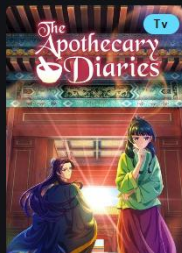
Clear History



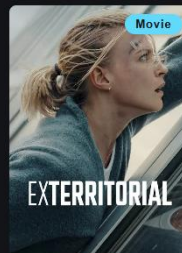
Final Destination ...
May 16, 2025



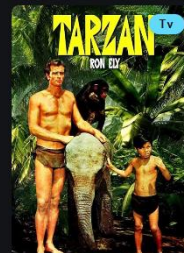
Havoc
May 8, 2025



The Apothecary D...
May 8, 2025



Exterritorial
May 8, 2025



Tarzan
May 8, 2025

MY WATCHLIST



Final Destination
Bloodlines

REMOVE



Havoc

REMOVE



The Siege

REMOVE



A Minecraft Movie

REMOVE



The Owl House

REMOVE